

머신러닝 프로젝트 (RISE) · 1주차

강의 소개와 개발환경 세팅

"15주 뒤, 여러분의 서버 폴더에는 예측 모델이 붙은 서비스가 하나 돌아가고 있을 것이다"

- 3단계 프로젝트: 바이브코딩 프로토타입 → ML 모델링·리더보드 경쟁 → ML 백엔드 통합 서비스
- 오늘: 과목 운영·평가 방식 → 바이브코딩이 무엇인가 → 서버 개발환경 접속·동작 확인 → 프로젝트 주제 준비
- 오늘의 결과물: `check_env.py` "모두 준비되었습니다" + 내 앱 주소에 뜬 첫 Streamlit 화면 캡처 + 주제 설문

학습목표와 오늘의 경로

강의 50분 + 서버 환경 실습 55분 + 주제 준비·정리 15분 — 설치할 것은 없다, 오늘은 "내 폴더에서 돌아가는 화면"이 전부다

학습목표

- 강의 전체 구성과 **3단계 프로젝트 운영·평가 방식**을 설명한다
- 바이브코딩**의 개념과 이 과목에서의 역할을 말한다
- 학과 서버 **JupyterHub**에 **내 계정으로 접속해** `~/work`에서 **Claude Code**와 **Streamlit**을 실행한다
- 개인 프로젝트 진행 방식과 좋은 주제의 조건을 알고 **설문에 응답**한다

시간	내용	형태
0:00	인사 · 오늘의 목표	강의 5분
0:05	PART 1 과목 소개 · 3단계 로드맵 · 평가 · AI 활용 원칙	강의 30분
0:35	PART 2 바이브코딩 개요 · 1단계에서 만들 것	강의 15분
0:50	PART 3 LAB 1-1~1-5 접속 · JupyterLab · Claude Code · 점검 · 첫 앱	실습 55분
1:45	PART 4 프로젝트 주제 준비 · 정리 · 과제	15분

실습 환경

준비물: **브라우저가 있는 기기**(노트북 권장 · 태블릿도 가능) + 인터넷 + **학번과 휴대전화 번호**(아이디 = 262mb+학번, 초기 비밀번호 = 휴대전화 번호 숫자만). 서버 주소 <https://ahnbi4.suwon.ac.kr:5300/>. 실습 파일은 서버 `~/work/shared/week1/`(`check_env.py` · `hello_streamlit.py` · `SETUP_GUIDE.md`)에서 복사.

01

PART 1

강의 소개와 3단계 프로젝트

무엇을 만들고, 어떻게 평가받는가 — 수업계획서를 한 번에

이 파트에서

- 1 이 과목은 무엇을 하는 과목인가
- 2 3단계 프로젝트 로드맵 (15주)
- 3 1단계 바이브코딩 5주 상세
- 4 평가 방식 · 학습자 AI 활용 원칙 · 운영 규칙

이 과목은 무엇을 하는 과목인가

머신러닝프로젝트(RISE) · 3학점(이론 1 · 실습 2) · 데이터과학부 3학년 · 김순찬 · 안홍렬 공동 담당 · 금 5~7교시 글경 507

한 문장으로

- 실제 데이터로 **문제를 정의**하고, **예측 모델**을 만들고, 그것을 **동작하는 서비스**로 구현하는 15주
- 강의 30% + 실습·프로젝트 70% — 듣는 시간보다 **만드는 시간**이 길다
- 교재 없음. 강의교안·실습 코드·양식은 LMS에 매주 게시
- 1단계(1~5주)는 **개인 프로젝트** — 각자 앱 하나. 2·3단계 운영 방식은 6주차에 담당 교수 안내

교과목표 (수업계획서)

- AI 코딩 도구로 웹·앱 프로토타입을 **신속하게 설계·구현**한다 → 1단계
- ML 개념·문제 정의를 이해하고 **전처리·피처 엔지니어링**을 수행한다 → 2단계
- 모델을 학습·검증하고 **평가지표·튜닝**으로 성능을 높인다 → 2단계
- 학습된 모델을 **REST API로 서빙**하고 프론트엔드와 통합한다 → 3단계
- 데이터 기반 서비스를 **기획·개발·발표**한다 → 전 단계 (1단계는 개인, 이후는 담당 교수 안내)

핵심역량은 **창의역량**, 전공역량은 **빅데이터기술역량**. 평가의 무게는 "얼마나 많이 아는가"가 아니라 "**문제를 스스로 정의하고 동작하는 것을 만들어 보여 주는가**"에 있습니다. 1주차부터 결과물이 쌓입니다.

3단계 프로젝트 로드맵 — 15주

같은 서비스가 3단계를 관통한다: 1단계에서 만든 화면에 2단계 모델이 들어가고, 3단계에서 API로 연결된다

1

1단계 · 1~5주 · 김순찬
바이브코딩 웹/앱 프로토타입

2

2단계 · 6~10주 · 안홍렬
ML 모델링 · 리더보드 경쟁

3

3단계 · 11~15주 · 안홍렬
ML 백엔드(REST API) + 프론트 통합

주차	내용	결과물	평가
1~5	강의 소개·환경 세팅 → 프롬프트 기반 개발 → 프로토타입 구현 → 개발 워크숍 → 시연·발표	서버에서 돌아가는 프로토타입(시연) · 기획서 · AI 활용 기록	1단계 20%
6~10	ML 개념·문제 정의·EDA → 전처리·피처 → 모델 학습·평가·튜닝 → 리더보드 경쟁 → 최종 제출·공유	학습 모델 · 리더보드 순위 · 접근법 발표	2단계 20%
11~15	모델 저장·서빙 개념 → REST API 구현·연동 → 통합 애플리케이션 → 프로젝트 개발 → 최종 발표	ML 통합 서비스 · 최종 발표·보고서	3단계 30%

왜 1주차에 환경 세팅인가

오늘 접속하는 서버 환경(JupyterLab · Python 3 · Claude Code · Streamlit)은 15주 내내 그대로 씁니다. 2단계의 scikit-learn은 이미 설치되어 있고, 3단계의 FastAPI가 같은 서버에 추가될 뿐입니다. 내 계정의 `~/work`가 15주의 작업 공간입니다.

1단계 바이트코딩 — 5주 상세

매주 120분 = 이론 + 실습(+워크숍/발표). 결과물이 다음 주의 입력이 된다

주차	수업 내용	결과물 (수업 후)	준비 (수업 전)
1 (오늘)	강의 소개 · 3단계 안내 · 바이트코딩 개요 · 서버 개발환경 접속·점검	환경 점검 캡처 · 주제 설문	브라우저 기기 · 학번 · 휴대전화 번호
2	프롬프트 기반 개발 · Claude Code 활용 · 요구사항→프롬프트 구조화	앱 2개 · 프롬프트 로그	Claude Code 기본 조작 예습
3	웹/앱 프로토타입 구현 · 화면 구성·기능 연결 · 실행·시연 준비	다중 화면 앱 · 시연 가능 상태 · 프로젝트 기획서	서비스 아이디어 구상
4	개인 프로젝트 개발 워크숍 · 짝 리뷰 · 교수 피드백	기능 명세 · 동작하는 프로토타입	개발 계획 · 기능 명세
5	갤러리 시연 · 라이트닝 · 동료 피드백 · 1단계 평가	시연 · AI 활용 기록 · 회고	시연 시나리오 · 최종 점검

주제는 **3주차 수업 후 확정**(기획서 제출). 그 전 2주는 공통 예제로 도구를 익히므로, 오늘 설문에 관심 주제와 데이터 후보를 적어 두면 3주차가 빨라집니다.

평가 — 상대평가, 다섯 항목

수업계획서 그대로. 3단계 프로젝트 합이 70%, 출석·참여가 30%

항목	비율	세부 기준	시점
출석	25%	학교 규정에 따라 성적에 반드시 포함	매주
1단계 프로젝트	20%	바이브코딩 결과물 시연·발표 평가 (루브릭 5항목 100점)	5주차
2단계 모델 성능 경쟁	20%	리더보드 기반 모델 성능 순위 경쟁	9~10주차
3단계 최종 프로젝트	30%	ML 백엔드 통합 서비스 개발 · 최종 발표	15주차
수업 참여	5%	출결 · 실습 참여도 · 짝 리뷰·동료 피드백	상시

1단계 루브릭 미리보기 동작·완성도 30 · 문제 정의·창의성 20 · 구현 이해도(Q&A) 20 · AI 활용 기록·정직성 15 · 발표·시연 15 (템플릿 04)

개인 점수 1단계는 개인 프로젝트 — 루브릭 100점을 그대로 20%로 환산. 배율 없음

자주 묻는 질문

"코드를 AI가 다 썼는데 점수를 받아도 되나요?" — 됩니다. 단, **본인이 데이터 흐름과 AI가 쓴 줄을 설명**할 수 있어야 하고(20점), **AI 활용을 기록**해야 합니다(15점). 이 두 항목이 1단계 점수의 35%입니다.

학습자 AI 활용 원칙 — 이 과목의 규칙

수업계획서: "책임 있는 AI 활용과 학문적 정직성". AI 활용 주차 1~15주 전부 ■사용

허용 — 그리고 반드시 할 것

- 코드 생성·수정·설명·테스트 작성·오류 해석에 AI를 자유롭게 쓴다
- 과제·프로젝트에 AI를 썼으면 **활용 사실을 명확히 표기**한다 (프롬프트 로그, 템플릿 03)
- 생성된 결과는 **읽고 · 실행하고 · 수정**한 뒤 제출한다
- AI가 틀렸던 사례를 기록한다 — 검증했다는 증거

금지 (수업계획서 명시)

- 학습자의 **수정·보완 없이** AI 생성물의 일부·전부를 그대로 제출
- AI가 만든 **허위 인용·출처**를 실제 자료처럼 제출
- 시험·과제·평가 중 교수자 **사전 허락 없는** AI 사용 → 부정행위
- (추가) 남의 프롬프트 로그·코드를 베껴 내 것으로 제출하는 것

2주차부터 **프롬프트 로그**를 남깁니다: 도구 · 목적 · 프롬프트 요약 · 어떻게 검증·수정했나 · 채택 여부. 10개 이내면 충분하고, "AI가 지어낸 함수 이름" 같은 실패 사례가 오히려 점수가 됩니다.

수업 운영 규칙과 소통

매주 브라우저 기기 지참 · LMS 사용 · 질문은 오류 메시지와 함께

항목	내용
수업 형태	금 5~7교시 연강 120분 운영(이론 → 실습/워크숍). 실습 시간에는 자리에서 직접 만든다 — 구경만 하면 다음 주에 못 따라온다
LMS	강의교안(PPT) · 실습 코드 · 양식 · 과제 제출 · 주제 설문 · 공지. 과제 마감은 다음 수업 전날 23:59
개발환경	학과 서버(JupyterHub · 개인 계정). 설치 없음 , 브라우저만 있으면 교내·교외 어디서나 같은 주소. 작업물은 서버 <code>~/work</code> 에 남는다
질문 방법	"안 돼요" 대신 오류 메시지 전체 + 무엇을 했는지 + 기대한 것 — 이 형식은 AI에게 물을 때도 똑같다(2주차)
면담	수업 후 글경 1012호 · 이메일(LMS 공지 참조) · 내선 031-229-8762
장애학생 지원	장애학생지원센터(031-229-8394) 또는 담당교수 면담으로 자료·과제·평가 지원 요청

알아 두기

서버 계정은 각자 것이지만 **서버와 Claude 토큰은 공용**입니다. `~/work` 밖은 로그아웃하면 지워지고, 사용 이력은 계정별로 남습니다. PART 3 "공용 서버 이용 규칙"에서 구체적으로 다룹니다.

02

PART 2

바이트코딩 개요

자연어로 의도를 말하고 AI가 코드를 쓰며, 사람은 실행 결과를 보고 방향을 잡는 개발 방식

이 파트에서

- 1 바이트코딩이란 — 전통 개발과 무엇이 다른가
- 2 도구 지형과 이 과목의 선택
- 3 1단계에서 만들 것 — 이탈 위험 체크 서비스
- 4 한계와 사람의 역할

바이브코딩(vibe coding)이란

Karpathy(2025.2): "코드가 존재한다는 사실을 잊고, 흐름(vibe)에 맡긴다" — 이 과목에서는 그 절반만 받아들인다

전통적 개발		바이브코딩 (이 과목의 정의)
사람이 쓰는 것	코드 한 줄 한 줄	무엇을 만들지(프롬프트) + 실행 결과에 대한 판단
한 바퀴	설계 → 코딩 → 실행 → 디버깅 (시간 단위)	프롬프트 → 생성 → 실행 → 오류 되돌려 주기 (분 단위)
첫 화면까지	반나절~며칠	10분 (오늘 LAB 1-5에서 확인)
병목	문법·API를 아는가	문제를 구체적으로 말할 수 있는가, 결과를 검증할 수 있는가
필요한 능력	언어·프레임워크 숙련	요구사항 구조화 · 코드 읽기 · 테스트 · 기록

실무에서 이렇게 틀린다 ①

"코드를 잊는다"는 시연까지는 통하지만, 5주차 Q&A(구현 이해도 20점)와 3단계 통합(남의 코드에 API 붙이기)에서는 통하지 않습니다. 그래서 이 과목의 바이브코딩은 **"AI가 쓰고, 내가 읽고 검증한다"** 입니다.

도구 지형과 이 과목의 선택

세 유형 중 이 과목은 ③ 에이전트 CLI(Claude Code)를 서버에서 쓴다 — 프롬프트는 어디에 넣어도 같다

유형	대표 도구	잘하는 것	이 과목에서
① 대화형 (웹)	ChatGPT · Claude · Gemini	기획 · 설명 · 오류 해석 · 짧은 코드	개인 계정이 있으면 보조로
② IDE 통합	VS Code + Copilot · Cursor	파일을 보며 수정 · 자동완성	쓰지 않음 (설치·개인 계정 필요)
③ 에이전트 CLI	Claude Code · Gemini CLI · Codex CLI	터미널에서 파일 생성·실행·테스트까지 자율 수행	기본 도구 — 서버 내 계정에 설정 완료

에디터 JupyterLab (브라우저) — 파일 브라우저 · 텍스트 편집기 · 노트북 · 터미널이 한 화면

AI 코딩 도구 Claude Code — 터미널에서 `claude`. 내 폴더의 파일을 읽고 만들고 실행한다

프로토타입 Streamlit — 파이썬 파일 하나로 웹 화면. 데이터과학부에 맞고 3단계 FastAPI와 자연 연결

실행·시연 `streamlit run` → **내 앱 주소**(로그인한 내 브라우저에서 열림). 시연은 자기 자리 화면으로 (3주차·5주차)

스택 질문

"내 노트북에 설치하면 안 되나요?" — 됩니다(2주차 이후 선택). 다만 수업·평가는 **서버 환경 기준**입니다. 전원이 같은 파이썬·같은 패키지·같은 주소 규칙을 써야 5주차 시연에서 "내 컴퓨터에서는 됐는데"가 사라집니다.

1단계에서 만들 것 — 예시 서비스 "이탈 위험 체크"

3주차에 함께 만드는 공통 예제. 화면 3개 + 규칙 기반 점수 함수 하나. 2단계 모델·3단계 API가 들어갈 자리가 미리 비어 있다

1

홈
서비스 소개 · 사용법

2

데이터 탐색
CSV 요약 · 이탈률 26.5%

3

이탈 위험 체크
입력 5개 → 점수·근거

4

(2단계) 모델 예측
규칙 → 학습 모델

5

(3단계) REST API
함수 호출 → HTTP 요청

core.py

3주차 LAB · 2단계에서 모델로, 3단계에서 API로 교체

```
def churn_score(contract, tenure, monthly, internet, support):
    """규칙 기반 0~100 위험 점수 - 지금은 '가짜 백엔드'"""
    score, reasons = 10, []
    if contract == "Month-to-month":
        score += 35; reasons.append("월 단위 계약 (+35)")
    if tenure < 6:
        score += 25; reasons.append("가입 6개월 미만 (+25)")
    if monthly >= 80:
        score += 15; reasons.append("월요금 80달러 이상 (+15)")
    ...
    level = "높음" if score >= 60 else "보통" if score >= 35 else "낮음"
    return {"score": score, "level": level, "reasons": reasons}
```

이 예제가 보여 주는 것

- 화면(Streamlit) 과 로직(core.py) 을 분리하면 로직만 갈아 끼울 수 있다
- 입력 5개 → 점수 하나: 2단계 "입력 → 예측값"의 형태와 같다
- 데이터: ~/work/shared/data/telco_churn.csv (통신사 고객 7,043명 · 21열)
- 주제가 없으면 이 앱의 **확장**이 기본 주제

내 프로젝트도 같은 뼈대입니다: 누가 · 무엇을 입력하면 · 무엇이 나오는가. 이 한 문장이 3주차 기획서의 첫 줄이고, 2단계에서 "예측값"이 되고, 3단계에서 API의 요청·응답이 됩니다.

한계와 사람의 역할 — AI가 못 하는 것이 곧 평가 항목

LLM은 다음 토큰을 확률로 고른다 → 그럴듯하지만 틀린 코드가 나온다 (2주차에서 원리를 다룬다)

AI 코딩 도구가 잘 틀리는 곳

- 존재하지 않는 함수·열 이름을 지어낸다 (내 데이터를 모르면 추측)
- 옛 버전 API를 쓴다 (학습 시점 이후 바뀐 것을 모른다)
- 한 번에 여러 가지를 시키면 앞부분을 잊는다
- 테스트 없이 "고쳤다"고 말한다 — 실행하면 그대로 깨져 있다
- 문제가 막연하면 막연한 코드가 나온다

그래서 사람이 하는 일 = 1단계 루브릭

- 문제 정의 — 대상·문제·기능·제약을 구체적으로 (문제 정의 20점)
- 검증 — 실행 · 읽기 · 테스트 (동작·완성도 30점)
- 설명 — 데이터 흐름과 AI가 쓴 줄을 내가 (구현 이해도 20점)
- 기록 — 프롬프트 · 검증 · 수정 (AI 활용 기록 15점)
- 보여 주기 — 3분 시연 + 교수 Q&A (발표·시연 15점)

오늘 LAB 1-5에서 Claude Code에게 첫 앱을 시켜 볼 때, 만들어 준 파일을 편집기로 열어 **한 줄씩 읽고** "이 줄이 무엇을 하는지" 스스로 말해 보세요. 2주차에는 그것을 AI에게 물어 보는 법(설명 프롬프트)을 배웁니다.

03

PART 3

개발환경 세팅 (서버)

LAB 1-1 → 1-5 순서대로. 끝나면 check_env.py가 "모두 준비되었습니다"를 출력하고, 내 앱 주소에 첫 화면이 뜬다

이 파트에서

- 1 이번 학기 개발환경 한 장 · 접속 정보(계정·비밀번호·폴더·주소)
- 2 LAB 1-1 접속·비밀번호 변경·~/work · 1-2 JupyterLab 세 도구
- 3 LAB 1-3 Claude Code · 1-4 환경 점검
- 4 LAB 1-5 첫 Streamlit 앱 · 공용 서버 이용 규칙 · 트러블슈팅

이번 학기 개발환경 — 한 장

내 컴퓨터에는 아무것도 설치하지 않는다. 브라우저 → 학과 서버 JupyterHub(내 계정) → 터미널에서 `claude · streamlit` → 내 앱 주소

1

① 브라우저
노트북 · 태블릿 · 폰

2

② JupyterHub 로그인
ahnbi4.suwon.ac.kr:5300 · 내 계정

3

③ 내 작업 폴더
~/work

4

④ 터미널
`claude · streamlit run`

5

⑤ 내 앱 주소
.../user/<아이디>/proxy/8501/

역할	노트북 설치형 (일반적 수업)	이번 학기 (학과 서버)
에디터	VS Code 설치 · 확장 설치	JupyterLab 텍스트 편집기 · 노트북 (브라우저)
AI 코딩 도구	Copilot·Cursor 개인 계정 · 로그인	Claude Code — 내 계정 터미널에서 <code>claude</code> (설정 완료, 로그인 절차 없음)
파이썬·패키지	Python 설치 · <code>venv</code> · <code>pip install</code>	Python 3 + 공통 패키지 설치 완료. 없는 것은 <code>pip install</code> — 내 계정에 유지
실행 화면	localhost:8501 (내 컴퓨터에서만 보임)	내 앱 주소 — 로그인한 내 브라우저에서 열림. 시연은 내 자리 화면으로
저장·백업	내 디스크 (노트북 고장 = 소실)	서버 ~/work (어느 기기에서 접속해도 같은 파일). ~/work 밖은 로그아웃 시 삭제

좋은 점의 대가

대가는 둘: **공용 서버**(3일마다 자동 로그아웃, ~/work만 남는다)와 **공용 토큰**(반 전체가 나눠 쓰고 계정별 사용 이력이 남는다). "~/work 안에서만 · 수업 용도로만 · 페이بل 모델 금지"가 이번 학기 개발환경의 규칙 전부입니다(PART 3 끝 "이용 규칙").

접속 정보 — 계정 · 비밀번호 · 폴더 · 주소

아이디 하나(수업코드+학번) · 폴더 하나(~/work) · 앱 주소 하나. 배경표는 없다

항목	값
주소	https://ahnbi4.suwon.ac.kr:5300/ — 교내·교외·휴대폰 모두 같은 주소 (https · 5300)
아이디	수업코드 262mb + 학번 — 예: 262mb22123456 (262mb = 김순찬 분반. 안홍렬 분반은 262ma)
초기 비밀번호	휴대전화 번호에서 하이픈(-)을 뺀 숫자 (예 01012345678) → 첫 접속 후 "비밀번호 변경" 필수
내 작업 폴더	~/work — 모든 파일은 여기에. 밖에 둔 파일은 로그아웃하면 삭제 (pip로 설치한 패키지는 유지)
공유 자료	~/work/shared(우리 분반) · ~/work/shared_all(전체 공통) — 읽기 전용, 고칠 파일은 내 ~/work로 복사
내 앱 주소	<a href="https://ahnbi4.suwon.ac.kr:5300/user/<아이디>/proxy/8501/">https://ahnbi4.suwon.ac.kr:5300/user/<아이디>/proxy/8501/ — check_env.py가 정확한 주소를 출력
Claude Code	터미널에서 <code>claude</code> — 계정마다 이미 설정. 계정·로그인 화면이 뜨면 손 들기

학번	아이디	작업 폴더	내 앱 주소	claude 실행 위치
22123456	262mb22123456	~/work	ahnbi4.suwon.ac.kr:5300/user/262mb22123456/proxy/8501/	~/work/<프로젝트> 안에서
21000789	262mb21000789	~/work	ahnbi4.suwon.ac.kr:5300/user/262mb21000789/proxy/8501/	15주 내내 같은 규칙

LAB 1-1 · 접속 · 비밀번호 변경 · 내 작업 폴더

로그인 → 비밀번호 변경 → 서버 시작 → 파일 브라우저에서 work. 첫 화면(홈)에 만든 파일은 로그아웃 후 사라진다 (5분)

① **접속·로그인** Chrome으로 <https://ahnbi4.suwon.ac.kr:5300/> → 아이디(262mb+학번) · 초기 비밀번호(휴대전화 번호 숫자만)

② **비밀번호 변경** 접속 화면의 "비밀번호 변경" 메뉴 → 새 비밀번호. 휴대전화 번호는 남이 알 수 있는 값

③ **서버 시작** 서버가 뜨면 JupyterLab 화면: 왼쪽 = 파일 브라우저(홈 ~), 가운데 = 런처 (Notebook · Console · Terminal · Text File)

④ **내 폴더** work 더블클릭 → 안에 shared · shared_all이 보이면 정상. 경로 표시가 work이면 성공

⑤ **표시 만들기** work 안에서 우클릭 → New File → 이름 README.md → 이름.학번 한 줄 저장(Ctrl+S) → 새로고침(⌘) 후 보이면 통과

폴더 구조

오늘 끝났을 때

```
~/
└─ work/
    ├── shared/
    │   ├── week1/
    │   │   └─ data/
    │   └─ shared_all/
    ├── README.md
    ├── week1/
    └─ churn_check/
```

- ← 홈 (로그아웃하면 work 밖은 삭제)
- ← 모든 작업은 여기 (유지)
- ← 우리 분반 공유 (읽기 전용)
- check_env.py · hello_streamlit.py
- telco_churn.csv ...
- ← 전체 분반 공통 (읽기 전용)
- ← 오늘 LAB 1-1
- ← 오늘 LAB 1-3~1-5
- ← 3주차 프로젝트 (예)

실무에서 이렇게 들린다 ②

홈(~)에 파일을 만들고 다음 주에 "사라졌다"는 학생이 매 학기 나온다. 파일 브라우저에서 work로 들어간 뒤 만들고, 터미널에서는 `cd ~/work/...` 를 먼저 친다. shared 안의 파일은 고치지 말고 복사한다(읽기 전용이라 저장 안 된다).

LAB 1-2 · JupyterLab 세 도구 — 터미널 · 편집기 · 노트북

런처(+)에서 셋을 열어 탭으로 나란히 둔다. 이 배치가 15주 기본 작업 화면 (10분)

도구	여는 법 · 쓰임	오늘 해 볼 것
터미널 (Terminal)	런처 → Terminal. 명령 실행 · <code>cd</code> · <code>streamlit run</code> . 프롬프트 끝 <code>~\$</code> = 홈	<code>cd ~/work</code> → <code>pwd</code> → 경로 끝이 <code>/work</code>
텍스트 편집기	파일 더블클릭 또는 런처 → Text File. <code>.py</code> · <code>.md</code> 편집, <code>Ctrl+S</code> 저장, 자동 저장 있음	README.md 열어 한 줄 추가 후 저장
노트북 (Notebook)	런처 → Python 3. 셀 단위 실행(<code>Shift+Enter</code>). 2단계 데이터 탐색에 주로 사용	셀에 <code>import pandas as pd; pd.__version__</code> 실행 → <code>work</code> 안에 저장
파일 브라우저	업로드 ↑ 아이콘 · 다운로드 우클릭 → Download · 새 폴더/파일 우클릭	<code>work/shared/week1/SETUP_GUIDE.md</code> 열어 보기

화면 배치 탭을 드래그해 오른쪽에 놓으면 좌우 분할. 추천: 왼쪽 편집기 · 오른쪽 터미널, 앱은 브라우저 새 탭

끊겨도 안전 브라우저 탭을 닫거나 노트북을 덮어도 서버의 파일·터미널·실행 중인 앱은 남는다. 다시 접속하면 그대로

3일 자동 로그아웃 서버 시작 후 3일이 지나면 자동 로그아웃 — 실행 중 앱·터미널은 종료, 파일은 `~/work`만 남는다. 다시 로그인해 서버 시작

실무에서 이렇게 틀린다 ③

터미널 복사·붙여넣기: 붙여넣기 `Ctrl+V`(안 되면 `Shift+Insert` 또는 우클릭 → Paste). 복사는 **드래그로 선택한 뒤** `Ctrl+C`. 선택 없이 `Ctrl+C`를 누르면 **실행 중인 프로그램이 종료**됩니다 — 앱이 갑자기 꺼졌다면 대개 이것.

LAB 1-3 · 터미널에서 Claude Code 시작하기

반드시 ~/work 안의 프로젝트 폴더에서 claude — Claude Code는 "지금 있는 폴더"를 프로젝트로 본다 (10분)

시작 계정마다 설정되어 있다 — 로그인·계정 관련 화면이 뜨면 손 대기

```
mkdir -p ~/work/week1 && cd ~/work/week1 # 오늘 폴더 만들고 들어가기
pwd # ../work/week1 인지 확인
claude # Claude Code 시작 - 전체 화면 UI
# 처음 뜨는 안내(테마·신뢰 확인)는 Enter 로 넘긴다
```

첫 요청 한 줄 요청 → 파일 생성 허락을 물으면 내용을 읽고 y

이 폴더에 sum10.py 를 만들어 줘.
파이썬으로 1부터 10까지의 합을 출력하고,
설명은 코드 주석으로만.

실무에서 이렇게 틀린다 ④

sum10.py가 홈(~) 에 생겼다면 cd를 빼먹고 claude를 실행한 것 — 로그아웃하면 사라진다. /exit → cd ~/work/week1 → 다시. 홈에 만든 파일은 mv ~/sum10.py ~/work/week1/ 로 옮긴다.

확인 만든 것은 항상 내가 실행해서 확인한다

```
# 파일 브라우저 work/week1 에 sum10.py 가 생겼다 → 편집기로 열어 읽는다
# 새 터미널을 하나 더 열어 실행:
cd ~/work/week1
python sum10.py
# 55 ← 나오면 통과. Claude 창으로 돌아가 /exit
```

조작	뜻
입력 후 Enter	요청 보내기. 여러 줄은 Shift+Enter(또는 W 뒤 Enter)
y / n	파일 생성·명령 실행 허락. 읽고 허락한다
Esc	진행 중인 작업 중단
/clear · /exit	새 대화 시작 · 종료 (Ctrl+C 두 번도 종료)
claude -c	방금 나간 대화를 이어서 (같은 서버 세션 안에서만)
/model	바꾸지 않는다 — 페이블(Fable) 모델 선택 금지

Claude Code — 2주차 전까지 알아 둘 것

터미널 안의 에이전트: 파일을 읽고·만들고·명령을 실행한다. 어디까지 허락할지는 내가 정한다

동작 방식

- **현재 폴더 = 프로젝트**. 그 안의 파일을 읽고 수정한다 → 프로젝트 폴더에서 실행하는 이유
- 파일 생성·수정·명령 실행 전에 **허락을 묻는다**. 무엇을 하려는지 보고 y
- 대화 이력은 `~/.claude(= ~/work` 밖)에 저장 → 로그아웃·3일 자동 로그아웃 뒤엔 `claude -c`가 안 될 수 있다. 남길 것은 **파일로**
- `CLAUDE.md`를 폴더에 두면 매번 읽는다 — 내 프로젝트 규칙을 여기에(4주차)
- `/help`로 명령 목록. 한국어로 말해도 된다

이 과목에서의 사용 규칙

- 요청은 **한 번에 하나** — "앱 만들고 테스트도 쓰고 배포도" 는 세 번으로
- 허락 화면에 `rm -rf · sudo · ~/work` 밖 경로가 보이면 n
- 생성된 파일은 편집기로 **열어서 읽고**, 터미널에서 **직접 실행**해 확인
- **공용 토큰** — 반 전체가 나눠 쓰고 계정별 사용 이력이 남는다. 짧고 구체적인 프롬프트, 불필요한 재요청 금지, 수업 외 용도 금지
- 프롬프트·결과·검증을 **프롬프트 로그**에 남긴다 (2주차부터)

한도 안내

Claude Code가 "rate limit"·"usage limit" 메시지를 보이면 공용 토큰 한도에 걸린 것입니다. 몇 분 뒤 다시 시도하고, 그동안 만들어진 코드를 읽거나 직접 고칩니다. 사용량이 큰 요청(전체 재작성·긴 파일 여러 개)은 수업 후로 미룹니다. 과도한 사용은 **계정별로 확인**되어 주의·이용 제한을 받을 수 있습니다.

LAB 1-4 · 실습 파일 복사 · 환경 점검 스크립트

내 폴더에서 check_env.py 한 번 — 설정 파일(.streamlit/config.toml)을 만들고 내 앱 주소를 알려 준다 (10분)

명령 한 줄씩 — 오류가 나면 그 줄에서 멈추고 메시지 전체를 복사

```
cd ~/work/week1          # 오늘 폴더 (LAB 1-3에서 만든 곳)
cp -r ~/work/shared/week1/* . # 복사 (끝의 . = 여기로)
ls                        # check_env.py hello_streamlit.py ...
python check_env.py
```

기대 출력 check_env.py (학번 22123456의 예)

```
=====
  서버 개발환경 점검 결과  (JupyterHub · 262mb22123456)
=====
Python 3.12.x                OK    CPython
패키지 streamlit             OK    1.64.0
패키지 pandas                OK    3.0.5
패키지 numpy                 OK    2.5.3
도구 claude                   OK    2.1.x (Claude Code)
작업 폴더                    OK    ~/work/week1 (~/work 안 - 유지)
공유 폴더                    OK    ~/work/shared · ~/work/shared_all
내 앱 주소                    OK
https://ahnbi4.suwon.ac.kr:5300/user/262mb22123456/proxy/8501/
설정 .streamlit/config.toml  OK    새로 생성 (port = 8501)
포트 비어 있음               OK    8501
=====
모두 준비되었습니다. 다음: streamlit run hello_streamlit.py → 브라우저 새 탭에 (위 앱 주소)
```

증상	원인	해결
작업 폴더 확인 필요	~/work 밖에서 실행	cd ~/work/week1 후 다시 python check_env.py
패키지 streamlit 확인 필요	내 계정에 미설치	pip install streamlit → 재실행 (내 계정에만 설치·유지)
포트 사용 중 주의	내 이전 앱이 아직 실행 중	그 터미널에서 Ctrl+C, 또는 pkill -f "streamlit run"
cp: cannot stat ...shared/week1	공유 폴더 이름·경로 오타	ls ~/work/shared 로 확인 후 다시

LAB 1-5 · 첫 Streamlit 앱 — 시키고, 읽고, 내 앱 주소로 연다

프롬프트 한 번 → streamlit run → 브라우저 새 탭에 내 앱 주소. 풍선이 뜨면 오늘 목표 달성 (15분)

프롬프트 (claude)

2주차에 배울 6요소가 이미 들어 있다

너는 Streamlit 개발자다.
 [목표] 환경 점검용 한 화면 앱: 제목, 파이썬·Streamlit 버전 표시, 이름 입력, "준비 완료" 버튼을 누르면 "OO님, 환경 준비 완료!" 성공 메시지와 풍선 효과
 [제약] 파일 하나(hello_streamlit.py), streamlit만 사용, 이름이 비면 경고. 기존 파일이 있으면 덮어써도 됨
 [출력] 코드만

실행

```
streamlit run hello_streamlit.py
# 터미널의 localhost:8501 은 서버 안 주소 - 열지 않는다
# 브라우저 새 탭: https://ahnbi4.suwon.ac.kr:5300/user/<아이디>/proxy/8501/
# (check_env.py 가 출력한 주소 그대로, 끝의 / 까지)
# 종료: 이 터미널에서 Ctrl+C
```

결과

shared/week1/hello_streamlit.py (검토·확정본)

```
import platform
import streamlit as st

st.title("머신러닝프로젝트(RISE) - 환경 점검")
col1, col2 = st.columns(2)
col1.metric("Python", platform.python_version())
col2.metric("Streamlit", st.__version__)

name = st.text_input("이름")
if st.button("준비 완료"):
    if not name.strip():
        st.warning("이름을 입력해 주세요.")
    else:
        st.success(f"{name}님, 환경 준비 완료! 2주차에 만나요.")
        st.balloons()
```

확인할 것

어떻게

내 앱 주소에서 열린다

내 계정으로 로그인한 브라우저에서만 열린다 — 옆 사람 브라우저에서는 안 열리는 것이 정상. 폰은 내 계정으로 로그인하면 열린다

성공 메시지 + 풍선

이름 입력 후 클릭 → 초록 메시지 + 풍선. 주소창이 보이게 캡처해 LMS 제출

코드 한 줄 읽기

st.metric, st.button, st.balloons — 각 줄이 화면의 어디에 해당하는지 짚지어 본다

공용 서버 이용 규칙 — 여섯 가지

여러 학생이 같은 서버.같은 Claude 토큰을 쓴다. 계정은 각자 것이지만 자원은 공용이다 — 사용 이력은 계정별로 남는다

규칙	어기면	방법
모든 파일은 <code>~/work</code> 안에	<code>~/work</code> 밖(홈 <code>~/tmp</code>)은 로그아웃하면 삭제 — 복구 불가	파일 브라우저는 <code>work</code> 로 들어간 뒤, 터미널은 <code>cd ~/work/...</code> 먼저
3일 자동 로그아웃에 대비	서버 시작 후 3일 → 자동 로그아웃: 실행 중 앱·터미널· <code>claude -c</code> 이력 종료	파일만 믿는다(커밋· <code>AI_LOG.md</code>). 시연 날 아침 다시 로그인 → 앱 실행
페이블(Fable) 모델 사용 금지	이용 제한 대상	Claude Code에서 <code>/model</code> 을 바꾸지 않는다 — 기본 모델 그대로
수업·프로젝트 용도로만 · 과도 사용 금지	공용 토큰 소진 → 반 전체가 멈춤. 계정별 사용 이력 확인 → 주의 조치·이용 제한	짧고 구체적인 프롬프트, "다시 해 봐" 반복 금지, 전체 재작성 자제
공유 폴더는 읽기 전용	<code>shared-shared_all</code> 안에서는 저장이 안 된다	내 <code>~/work</code> 로 복사해서 사용 (<code>cp -r</code>)
비밀번호 변경 · 계정 공유 금지	초기 비밀번호(휴대전화 번호)는 추측 가능 — 남이 내 토큰을 쓴다	첫 접속 때 "비밀번호 변경". 아이디·비밀번호를 남에게 주지 않는다

내 프로세스·폴더

내 계정 안에서는 마음껏 — 밖은 없다

```

kill -f "streamlit run"      # 내 앱 전부 종료 (내 계정 프로세스만 죽는다)
ps -ef | grep streamlit      # 남아 있나 확인
du -sh ~/work                # 내 작업 폴더 크기 (대용량 파일은 넣지 않는다)

```

AI에게도 같은 규칙

Claude Code 허락 화면에서도 같은 규칙: 명령에 `rm -rf-sudo-~/work` 밖 경로가 보이면 n. "이 폴더 밖은 건드리지 마"를 첫 요청에 넣어 두면 대부분 예방됩니다(4주차 CLAUDE.md).

트러블슈팅 — 오늘 가장 많이 나오는 여덟 가지

해결보다 중요한 것: 오류 메시지 전체를 복사하는 습관. 2주차에 그 메시지를 AI에게 넘긴다

증상	원인	해결
로그인이 안 됨	아이디 형식(262mb+학번) · 비밀번호에 하이픈 · 바꾼 비밀번호를 잊음	형식 확인 · 숫자만 · 잊었으면 강사에게(초기화)
"서버 시작" 화면이 뜸	3일 자동 로그아웃 뒤 첫 접속	시작 버튼 → 잠시 대기 → JupyterLab
내 앱 주소가 안 열림 (404 · 빈 화면 · 403)	앱이 안 떠 있음 / 주소 오타 / 로그인 안 된 브라우저	앱 터미널 확인 · <code>check_env.py</code> 출력 주소를 그대로(끝 / 포함) · 내 계정으로 로그인한 브라우저에서
Port 8501 is already in use	내 이전 앱이 아직 실행 중	그 터미널 탭 <code>Ctrl+C</code> , 또는 <code>pkill -f "streamlit run"</code>
ModuleNotFoundError: No module named ...	내 계정에 미설치	<code>pip install <패키지></code> 후 재실행 (유지됨)
claude가 대기."usage limit"	공용 토큰 한도	몇 분 후 재시도 · 프롬프트를 짧고 구체적으로
어제 만든 파일이 없다	<code>~/work</code> 밖에 저장한 뒤 로그아웃	복구 불가 — 항상 <code>~/work</code> 안에
붙여넣기가 안 됨	브라우저 클립보드 권한	<code>Shift+Insert</code> · 우클릭 → Paste · Chrome 클립보드 허용

수업 끝까지 해결이 안 되면 **오류 메시지 캡처 + 어느 단계(LAB 1-x)인지**를 LMS 질문 게시판에 올립니다. `SETUP_GUIDE.md`(내 폴더에 복사됨)에 같은 표가 있으니 집에서 재현할 때 참고하세요.

04

PART 4

프로젝트 주제 준비

1단계는 개인 프로젝트 — 주제는 3주차에 확정되지만, 오늘 설문이 그 출발점이다

이 파트에서

- 1 개인 프로젝트 진행 방식
- 2 좋은 주제의 조건 4개 — 미리 보기
- 3 주제 설문 (LMS, 오늘 제출)

개인 프로젝트 진행 방식 — 혼자서 다섯 가지 일을 한다, 그래서 작아야 한다

49명 혼자 앱 하나. 화면·로직·데이터·운영·발표를 한 사람이 하므로 범위가 곧 성패다

진행 원칙

- **혼자 만든다** — 2주차부터 5주차까지 ~/work 안의 내 프로젝트 폴더 하나, 앱 하나
- 범위는 **Must 기능 3개 · 화면 3개** (3주차 조건 4개로 거른다)
- **짹(옆 사람)**과 매주 10분 서로 앱을 열어 보고 피드백 — 4주차 짹 리뷰·짹 데모, 5주차 갤러리 시연
- 주제가 없으면 **공통 예제(이탈 위험 체크)의 확장**이 기본 주제
- 2·3단계 운영 방식(개인/팀)은 6주차에 안홍렬 교수 안내

하는 일

1단계에서 (전부 내가)

화면	와이어프레임 · Streamlit 페이지 · 레이아웃
데이터·로직	데이터 확보 · <code>core.py</code> 함수 · 2단계 예측 연결 설계
운영·문서	시연용 앱 실행 · README · 프롬프트 로그
발표	기획서 · 시연 시나리오 · 3분 시연 · 백업 영상

실무에서 이렇게 틀린다 ⑤

혼자 하니까 **AI에게 시키는 양이 많아진다** — 그래서 프롬프트 로그와 "내가 읽고 검증했다"는 증거가 더 중요합니다. 5주차 Q&A에서 교수가 AI가 쓴 줄을 골라 "이 줄은 무슨 뜻?"을 묻습니다(구현 이해도 20점).

좋은 주제의 조건 4개 — 미리 보기

3주차에 이 표로 내 주제를 거른다. 오늘은 "내 관심 주제가 통과하는가"만 생각해 본다

조건	질문	예 (통과 / 탈락)
① 5주차에 시연 가능	Must 기능 3개를 4주차 안에 혼자 만들 수 있나? 외부 계정·승인이 필요한가?	통과: 공개 CSV 기반 추천 / 탈락: 실시간 결제·로그인 필수
② 데이터가 있다	공개 데이터·직접 입력·무료 API 중 무엇인가? 지금 당장 받을 수 있나?	통과: 공공데이터포털·Kaggle·UCI / 탈락: "학교에서 받을 예정"
③ 예측 기능이 붙는다	"입력 → 예측값" 하나를 말할 수 있나? (2단계 모델, 3단계 API)	통과: 매물 정보 → 월세 예측 / 탈락: 단순 게시판
④ 사용자가 구체적	"누가, 언제, 왜" 쓰는지 한 사람으로 말할 수 있나?	통과: 자취 2년차 식비 관리 / 탈락: "모든 사람"

주제 예 학식 혼잡 예측 · 중고 노트북 가격 추정 · 자취 식비 관리 · 강의평 요약 · 통신사 이탈 관리(공통 예제 확장)

데이터 출처 예 공공데이터포털(data.go.kr) · 서울열린데이터 · Kaggle · UCI · 직접 수집 · 수업 제공 데이터 6종 (~/work/shared/data/)

아이디어의 크기가 아니라 **"구체적인 한 사람의 문제"**가 점수를 만듭니다. 2주차 LAB 2-2에서 이 아이디어 하나로 첫 프롬프트를 씁니다.

주제 설문 — 오늘 수업 후 LMS 제출

5문항 · 3분. 응답을 보고 2주차에 겹치는 주제·데이터를 묶어 안내한다

#	문항	답 예
1	관심 주제 2개 (조건 4개를 떠올리며)	"학식 혼잡 예측", "중고 노트북 가격 추정"
2	쓸 수 있을 것 같은 데이터 (출처·형태)	공공데이터포털 CSV / 직접 설문 / 없음(공통 예제)
3	파이썬 자신감 (1~10) · 써 본 도구	6 · pandas, matplotlib / Claude Code 처음
4	짜을 원하는 사람 (1명, 선택)	이름·학번 — 없으면 옆자리
5	오늘 환경 세팅 결과	전부 OK (내 앱 주소에서 풍선 확인) / 막힌 단계: LAB 1-x, 오류 메시지 캡처 첨부

운영

4번(짜)은 참고만 합니다 — 파이썬 자신감 상·하를 짝으로 묶는 것이 우선. 5번의 "막힌 단계"는 2주차 첫 10분 트러블슈팅에 씁니다.

05

PART 5

정리와 과제

오늘 배운 것을 한 표로, 그리고 다음 주까지 할 것

이 파트에서

- 1 한 장 요약
- 2 셀프 체크 5문항
- 3 과제: 환경 캡처 · 설문 · 연습
- 4 2주차 예고 — 프롬프트 기반 개발과 Claude Code

한 장 요약

오늘의 표 하나 — 나머지 14주는 이 표의 각 칸을 채우는 시간

주제	핵심	오늘 어디서
3단계 프로젝트	프로토타입(1~5) → 모델·리더보드(6~10) → API 통합(11~15). 같은 서비스가 관통	PART 1 로드맵
평가	출석 25 · 1단계 20 · 2단계 20 · 3단계 30 · 참여 5. 1단계 루브릭은 동작·문제·이해·기록·발표	PART 1 평가
AI 활용 원칙	자유롭게 쓰되 표기·수정·검증. 그대로 제출·허위 출처 금지	PART 1 원칙
바이브코딩	AI가 쓰고, 내가 읽고 검증한다. 병목은 문법이 아니라 "문제를 구체적으로 말하기"	PART 2
개발환경	브라우저 → JupyterHub(ahnbi4.suwon.ac.kr:5300, 262mb+ 학번) → ~/work → claude · streamlit run → 내 앱 주소(.../proxy/8501/)	PART 3 LAB 1-1~1-5
이용 규칙	~/work 안에만 저장 · 3일 자동 로그아웃 · 페이블 모델 금지 · 수업 용도만(계정별 사용 이력) · 공유 폴더 읽기 전용 · 비밀번호 변경	PART 3
프로젝트	개인 · Must 3개·화면 3개 · 3주차 확정 · 조건 4개(시연·데이터·예측·사용자) · 오늘 설문	PART 4

셀프 체크 — 5문항

옆 사람과 1분씩. 답은 오늘 슬라이드 안에 있다

- **Q1.** 3단계 프로젝트 각각의 결과물과 평가 비율을 말하라.
- **Q2.** 1단계 루브릭에서 "AI가 코드를 다 썼어도" 점수를 좌우하는 두 항목은?
- **Q3.** 학번이 22123456이면 아이디·작업 폴더·내 앱 주소는 각각 무엇인가? 그 주소는 누가 열 수 있나?
- **Q4.** `streamlit run`을 쳤더니 `Port 8501 is already in use`가 떴다. 가장 먼저 할 일은? 홈(~)에 만든 파일은 왜 위험한가?
- **Q5.** `claude`를 실행하기 전에 반드시 하는 일 하나는? 왜?

정답 요지 — Q1 프로토타입 20% · 모델 순위 20% · 통합 서비스 30% / Q2 구현 이해도(Q&A) 20 · AI 활용 기록 15 / Q3 262mb22123456 · ~/work · <https://ahnbi4.suwon.ac.kr:5300/user/262mb22123456/proxy/8501/> — 내 계정으로 로그인한 브라우저만 / Q4 내 이전 앱 터미널에서 `Ctrl+C`(또는 `kill -f "streamlit run"` — 내 계정 것만 죽는다); ~/work 밖은 로그아웃하면 삭제 / Q5 `cd ~/work/<프로젝트>` — Claude Code는 현재 폴더를 프로젝트로 보고 파일을 거기에 만든다

과제 — 다음 주 수업 전까지 (개인)

LMS 제출. 2주차 첫 10분에 막힌 단계를 함께 푼다

#	과제	제출물	기준
1	환경 세팅 완료 증빙	check_env.py 결과 캡처 + 내 앱 주소에 뜬 hello_streamlit.py 풍선 화면 캡처(주소창 포함)	"모두 준비되었습니다" + 성공 메시지
2	주제 설문	LMS 설문 5문항	관심 주제 2개는 조건 4개를 의식해서
3	(예습) Claude Code 기본 조작	~/work의 내 폴더에서 claude에게 코드 요청 1 회 → 실행 결과 캡처	"1~10 합"이든 무엇이든, 내가 실행까지
4	(선택) SETUP_GUIDE.md 집에서 재현	집·카페·폰에서 같은 주소로 로그인해 앱 실행	교외 접속 확인 = 5주차 시연 리허설 (3일 자동 로 그아웃도 겪어 본다)

다음 주

2주차 예고: **프롬프트 기반 개발과 Claude Code** — 도구 지형과 LLM 동작 원리 · 프롬프트 6요소(역할·목표·맥락·제약·출력·예시) · 생성→실행→수정 루프 · 생성 코드 검증. 실습 6개 : 나쁜/좋은 프롬프트 비교 → 첫 앱 → 일부러 깨진 앱 고치기 → 기능 추가 → 테스트 생성. 오늘의 서버 환경을 그대로 씁니다.

- 참고: 수원대 2026-2 수업계획서 「머신러닝프로젝트(RISE)」 · Karpathy, "vibe coding" (X, 2025.2) · Streamlit Docs "Get started" · Claude Code Docs (docs.anthropic.com) · JupyterLab User Guide